My News

CS 403: AI-powered news service

Assigned: October 9, 2026 **Points:** 100
Due: Friday, October 23, 2026, at 11:59 p.m. Eastern
Deliverable: A Flutter MVP ready for a limited commercial beta.
Presentation: Physical phone through Phone Link, live in class or by video.

The challenge

News organizations often cover the same event, but readers must visit several websites to compare their coverage and understand the background. Your task is to build **My News**, an application that organizes coverage of the same event into one story and lets readers explore its sources, background, and AI-generated summaries.

The reader opens a Flutter app, sees a list of top stories, and selects one. The story screen presents an overview and relevant buttons: original news sources, background resources, and AI summaries that explain different perspectives. For example, a story covered by Fox News, BBC, and NPR appears once in the story list, with separate buttons for each outlet's coverage.

This project uses AI in two ways: **you will use AI to help develop the software, and your software will use AI to help readers understand the news.** You remain responsible for understanding, testing, and explaining the result.

Learning objectives

By completing this project, you will be able to:

- Build a Flutter interface that consumes your own server API.
- Deploy a backend that collects, stores, and processes external data on a schedule.
- Distinguish duplicate articles from different articles covering the same event.
- Generate and reuse AI summaries grounded in identifiable sources.
- Handle stale data, unavailable services, and offline use.
- Explain and evaluate AI-assisted development decisions.

1. Required architecture

The server is the shared source of processed news. It collects news feeds and relevant background information, groups related articles, generates summaries, and stores the results. The Flutter app requests those stored results through your API.

Component	Responsibilities
News and background sources	Supply article metadata, available text, and relevant context.
Scheduled server worker	Fetch sources every hour, normalize records, group articles, update AI summaries, and remove expired content.
Server database	Store articles, story groups, background links, summaries, and processing timestamps.

Component	Responsibilities
Server API	Return stored story lists and story details to all clients.
Flutter app	Display stories and buttons, retrieve server results, and optionally cache them locally.

The Flutter app must not perform news ingestion, story grouping, or AI generation. Opening the app, refreshing its screen, or pressing an AI-summary button must retrieve shared results rather than start a new upstream news or AI request. Opening an original publisher or background link is allowed.

Choose your backend language, framework, database, hosting provider, and AI service. Deploy the backend to an internet-accessible service; a server running only on your laptop is not sufficient. Use persistent storage that survives an application restart. Keep provider credentials on the server and out of the Flutter build and submitted code.

The goal is to demonstrate an architecture in which many users reuse the same work. You do **not** need to support or load-test millions of users.

2. Server collection and retention

Implement the following:

1. **At least three distinct news publishers.** Use RSS feeds, news APIs, or other supported access methods. One aggregator may provide multiple publishers. The starter app's publishers are examples, not mandatory choices.
2. **Automatic hourly collection.** The deployed schedule must run even when no one has the app open. An instructor-triggered collection command may also be provided for demonstrations; protect it from general users.
3. **Duplicate prevention.** Re-fetching the same article must update or reuse its record rather than create another copy. Store a stable identifier or normalized source URL.
4. **A rolling seven-day window.** Use article publication time, not download time. On each hourly run, remove articles older than 168 hours. API responses must exclude expired articles between runs. Document how you handle missing or invalid publication dates.
5. **Consistent cleanup.** Remove story groups with no remaining articles and their associated summaries and background associations. For groups that still have recent articles, rebuild affected summaries from retained material; do not serve a summary that still depends on removed articles. Retain background resources only while linked to an active story. Background publications may themselves be older than seven days.
6. **Failure handling.** A failed feed or AI request must not crash the whole collection run or erase still-valid results. Record successes and failures, show the last successful collection time, and mark unavailable or outdated summaries clearly.

Store timestamps consistently, such as in UTC, and display readable dates in the app. Record enough job information to demonstrate the schedule, including start time, outcome, and article counts. Operational logs may outlive the news window, but must not retain expired article bodies or summaries.

3. Group articles into stories

An **article** is one publisher's report. A **story** is an event that may have several articles from different publishers.

Automatically assign related articles to the same story. You may use keywords, entities, time windows, similarity measures, AI, or a combination. Matching one keyword is not enough: two unrelated events involving the same person should not automatically become one story.

- Preserve the individual publisher, title, publication time, URL, and available text for each article.
- Give each story a stable ID, a readable headline, and an update time.
- Keep single-source stories when no matching coverage is found; label the number of sources accurately.
- Support new coverage joining an existing story without duplicating it in the list.
- Explain your grouping method and at least one situation in which it might make a mistake.

For evaluation, provide at least **five example story groups with two or more publishers each**, plus **two examples of related topics that should remain separate stories**. Use live data when available. A clearly labeled fixture dataset may demonstrate difficult cases, but the application must also ingest live news.

4. Story screens and buttons

Build the client in **Flutter and Dart**. Demonstrate a native build running on your physical phone, displayed through Microsoft Phone Link. An emulator may be used during development, but does not replace the required phone presentation. Android is sufficient; both platforms and app-store publication are not required. Verify your phone and computer support the screen-display workflow you will use before presentation day. A PWA, a plain website, or a Flutter wrapper around the starter webpage does not satisfy the client requirement. Flutter web may be an additional deliverable.

Story list: Display one entry per story, including its headline, latest article time, and number of distinct publishers. Offer a refresh action that retrieves your server's current data. Define your "top stories" ordering; newest first is acceptable for this MVP. Show loading, empty, and error states.

Story detail: Display an overview and clearly separated button groups:

Button group	Required behavior
Original coverage	One labeled button per available publisher. Open that publisher's article or a list of its articles for the story.
Background	Open relevant context, such as a Wikipedia entry, official report, historical explanation, or primary document. Labels should describe the resource.
AI overview	Show a concise synthesis of the retained coverage, with links or references to the supporting articles.
AI perspectives	Show distinct, clearly named viewpoints that the available material actually supports.

Do not show buttons for sources that did not cover the story. Automatically retrieve or select background resources on the server using story information; document the method. Demonstrate relevant background links for at least three stories. If no useful resource is found, display "No background resource available" rather than an unrelated link.

Original wording and AI wording must be distinguishable. Publisher buttons should let readers reach the original wording directly. In-app excerpts or full text are optional and must use content your chosen source permits you to display. Do not label an excerpt as a full article or an AI rewrite as a verbatim quotation. Full-text copying and paywall bypass are not requirements.

5. AI summaries and perspectives

The backend must use an actual AI model to generate summaries from collected material. Hard-coded summaries or word substitutions do not satisfy this requirement.

For each processed story, provide:

- **Overview:** A short explanation of what happened, identifying agreement, disagreement, and uncertainty where relevant. Agreement among sources is not proof that a claim is true.
- **Perspectives:** At least two distinct viewpoints when supported by the evidence. Examples include supporters and critics, residents and officials, or economic and environmental concerns. Political perspectives may be used when relevant and supported; do not assign viewpoints solely from a publisher's name.
- **Traceability:** Identify which articles support each summary or viewpoint. Show when the AI output was generated and label it "AI-generated." If input consists only of feed excerpts, disclose that limitation.

Do not invent opposition for a story that has no meaningful disagreement. "Insufficient evidence for distinct perspectives" is an acceptable result for that story. Demonstrate at least three stories for which meaningful contrasting perspectives are supported.

Generate and store summaries as part of backend processing, then reuse them for all readers. Regenerate when relevant inputs change, including expiration of supporting articles. Repeated reads of unchanged content must not call the AI model again. Treat collected article text as data rather than instructions to the model.

Save the prompts and model configuration in the project. Manually check three generated summaries against their inputs and report unsupported claims, missing context, and corrections. If generation fails, show an honest status instead of presenting fabricated fallback text as AI output.

6. API and local storage

Provide documented endpoints for a story list and a complete story detail. For example, `GET /stories` and `GET /stories/{id}` may return JSON containing IDs, timestamps, sources, background links, summary text, and summary status. Equivalent routes are acceptable.

Local caching is **optional**. If implemented, cache server-produced content, display the last synchronization time, indicate offline mode, and hide expired news even while offline. Explain that publisher pages may require a network connection. The server remains responsible for collection and processing.

7. Using the supplied starter project

The attached `myNews.zip` contains `index.html`, `style.css`, `app.js`, and a README. Its JavaScript loads Fox, NPR, and BBC feeds through browser-accessible proxies, saves individual articles in browser local storage, filters articles, and prunes older items.

Use it as a reference for behavior and data fields. Move the collection and retention concepts to your backend and implement a new Flutter interface. The starter's duplicate detection does not group different publishers' coverage of the same event. Its displayed paragraphs come from feed content; the supplied

implementation does not perform AI-model summarization. Your project must add those capabilities.

8. Two-week work plan

Target	Milestone
October 9–11	Choose technologies, inspect the starter, sketch the Flutter screens, define records and API responses, and deploy a minimal backend.
October 12–14	Collect from three publishers, persist records, prevent duplicates, and implement scheduled collection and expiration.
October 15–16	Connect Flutter to the server and demonstrate story grouping and publisher buttons. **Suggested checkpoint: October 16.**
October 17–19	Add background resources, AI overviews, supported perspectives, and reusable server-side summaries.
October 20–22	Test failures and retention, review AI output, refine the interface, and prepare documentation and demonstration.
October 23	Submit the complete project by 11:59 p.m. Eastern.

Prioritize a complete working path through the application. Accounts, personalization, notifications, advertising, payments, and large-scale infrastructure are outside the required scope.

MVP scope and commercial beta readiness

Deliver an **MVP (minimum viable product)**: the smallest complete, useful version of My News that satisfies the required workflow. Keep the feature set focused, but make that workflow reliable enough for a limited commercial beta release to real users. An MVP is not an unfinished demonstration with broken essential features.

The release candidate must meet these acceptance criteria:

- **Complete core workflow:** A reader can launch the app, load stories, select a story, use source and background buttons, view available AI analysis, and return to the list without dead ends.
- **No known critical or major defects:** Resolve reproducible crashes, data loss or corruption, exposed credentials, broken core navigation, and failures that prevent required features from working. Minor cosmetic issues may remain only if documented and they do not impair use.
- **Honest, reliable information:** Publisher labels and links match their articles; AI text is labeled; expired content is not served; unavailable information is explained. Fix known unsupported AI claims in the release candidate and document the remaining limitations of automated summaries.
- **Graceful failure:** Test slow or unavailable networks, unavailable providers, empty results, and AI failures. Show readable status messages and recovery actions rather than an endless spinner or raw error output.
- **Usable phone interface:** Text is legible, buttons are easy to tap, scrolling and back navigation work, and required content is not clipped on the presentation phone.
- **Repeatable release:** Install and launch a release build on the phone, test against the deployed backend, and record the build/version, device, test date, and results. Complete three consecutive runs of the core workflow without a blocking defect.

- **Release notes:** Include a short feature list, known minor issues, service limitations, and a way for beta testers to report problems. Maintain a defect log listing severity, steps to reproduce, status, and verification of fixes.

This is a beta-readiness standard, not a claim that software can be proven entirely bug-free. Public launch, app-store approval, paid accounts, and production-scale traffic testing are not required. Unresolved critical or major defects mean the MVP is not beta-ready and reduce credit in the affected rubric categories.

9. Submission

Submit one project package or repository link containing:

1. **Source code:** Flutter client, backend, database setup or migrations, scheduled-job configuration, and AI prompts. Include an example environment file with placeholders, never real secrets.
2. **Runnable client:** An Android APK or an iOS demonstration with reproducible build/run instructions. Include the deployed API URL and keep the service available for grading.
3. **README:** Setup instructions, architecture diagram, data model, API description, grouping method, retention behavior, service choices, expected operating costs or quotas, and known limitations, release notes, and defect log.
4. **A 5–7 minute presentation, live in class or by recorded video:** Present the app running on your physical phone, with its screen displayed on your computer using Microsoft Phone Link. A desktop-only or emulator-only demonstration does not meet this requirement. Rehearse Phone Link, make text readable, and suppress personal notifications. For the live option, include presentation slides or a demonstration checklist in your submission; for the video option, include a playable video file or accessible link. Demonstrate the real deployed service rather than only mock data. Show the story list, a grouped story, publisher and background buttons, AI views, server deployment, and evidence of scheduled collection. Explain how two clients reuse the same stored results.
5. **Verification evidence:** A short table of the checks below with actual results, plus screenshots or logs where useful. Include your grouping examples and review of three AI summaries.
6. **AI development reflection:** In approximately one page, name the AI development tools used, give two representative prompts, explain one AI-generated mistake you found and corrected, and describe what you personally verified. For a team submission, identify each member's contributions.

10. Required verification checks

Check	Expected result
Repeated ingestion	Fetching identical inputs twice does not duplicate articles or stories.
Same event, different publishers	Relevant articles appear under one story with working publisher buttons.
Similar topic, different events	Unrelated events remain separate.
Hourly automation	Deployed configuration and at least two consecutive scheduled-run records show collection without client activity.
Seven-day boundary	Controlled timestamps demonstrate that expired articles leave the API and are removed during cleanup; recent articles remain. No seven-day waiting period is necessary.

Check	Expected result
Source expiration inside a group	Expired source buttons disappear and summaries no longer depend on removed content.
Shared AI output	Repeated reads from two clients reuse the same stored summary version without additional generation calls.
Provider failure	An unavailable news source or AI model produces a clear status while valid stored stories remain usable.
Client connectivity failure	The app shows a useful retry state or clearly labeled cached content.
Physical-phone presentation	The release build runs on the phone and is visibly presented through Phone Link, live or in a recorded video.
Beta-readiness regression	Three consecutive core-workflow runs pass, and the defect log has no unresolved critical or major issues.
Source fidelity	Original excerpts remain original; AI claims and perspectives can be traced to their inputs.

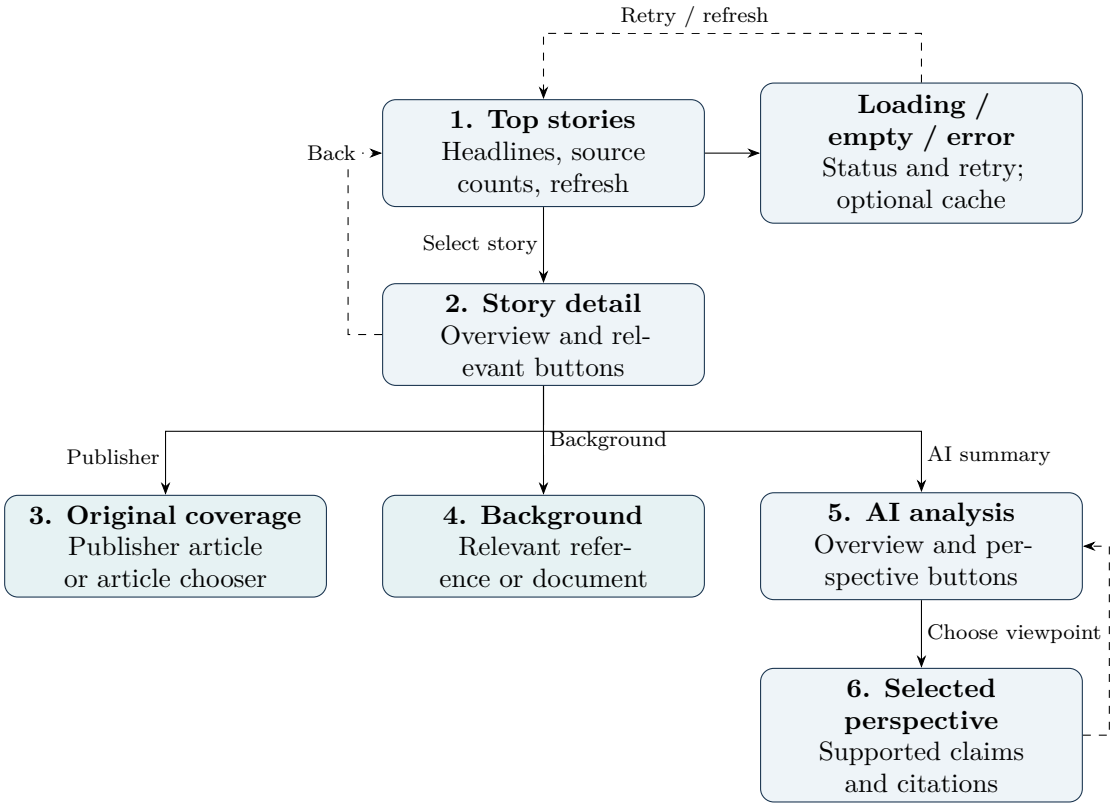
11. Grading rubric

Category	Points
Deployed backend, persistent storage, API, and server/client separation	15
Three-publisher ingestion, hourly scheduling, duplicate handling, and seven-day retention	20
Automatic story grouping and preserved source associations	15
Flutter story list, detail screen, source/background buttons, usability, and beta readiness	20
Grounded AI summaries, supported perspectives, attribution, and shared reuse	20
Verification evidence, documentation, demonstration, and AI reflection	10
Total	**100**

Credit is based on demonstrated behavior and the quality of your explanation. Screenshots or mock data alone do not demonstrate a working deployed system. A successful submission lets a reader select one story, inspect multiple sources and useful background, and read clearly labeled AI analysis—all served from shared backend processing.

Appendix A. Screen flow

This is a suggested navigation model. Equivalent designs are acceptable when all required behavior is present. Solid arrows show navigation; dashed arrows show recovery or return behavior.



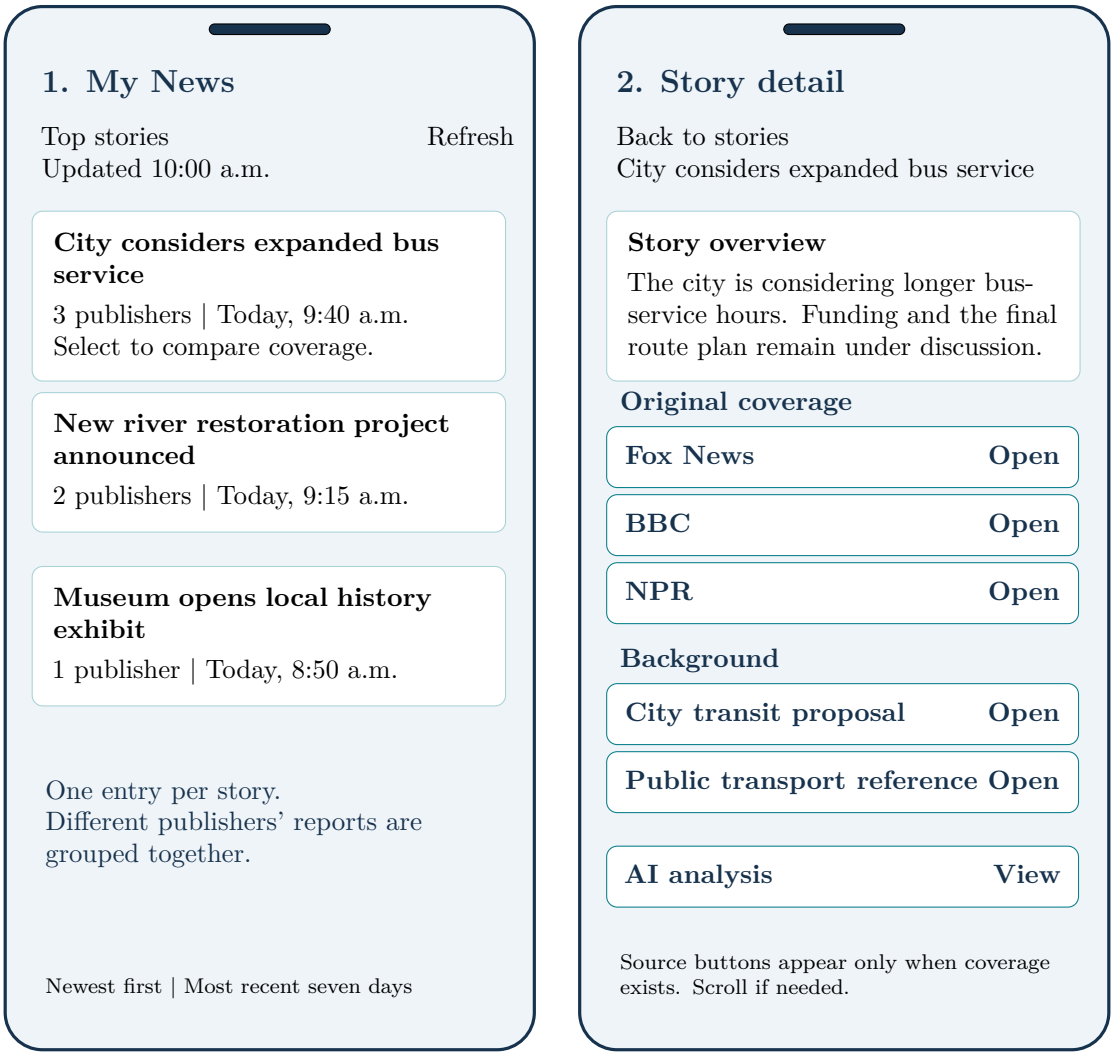
Return behavior: Each detail or AI screen has a visible Back action. Original coverage and background resources may open in a browser; returning to My News preserves the selected story. Multiple articles from one publisher may use an intermediate chooser.

Server boundary: Story and AI screens read stored server results. Pressing these buttons does not trigger news collection or AI generation. The hourly worker operates independently of this screen flow.

Unavailable content: Explain missing background, unsupported perspectives, or a failed AI summary. Show only source buttons actually associated with the story.

Appendix B. Mockups: browsing and story detail

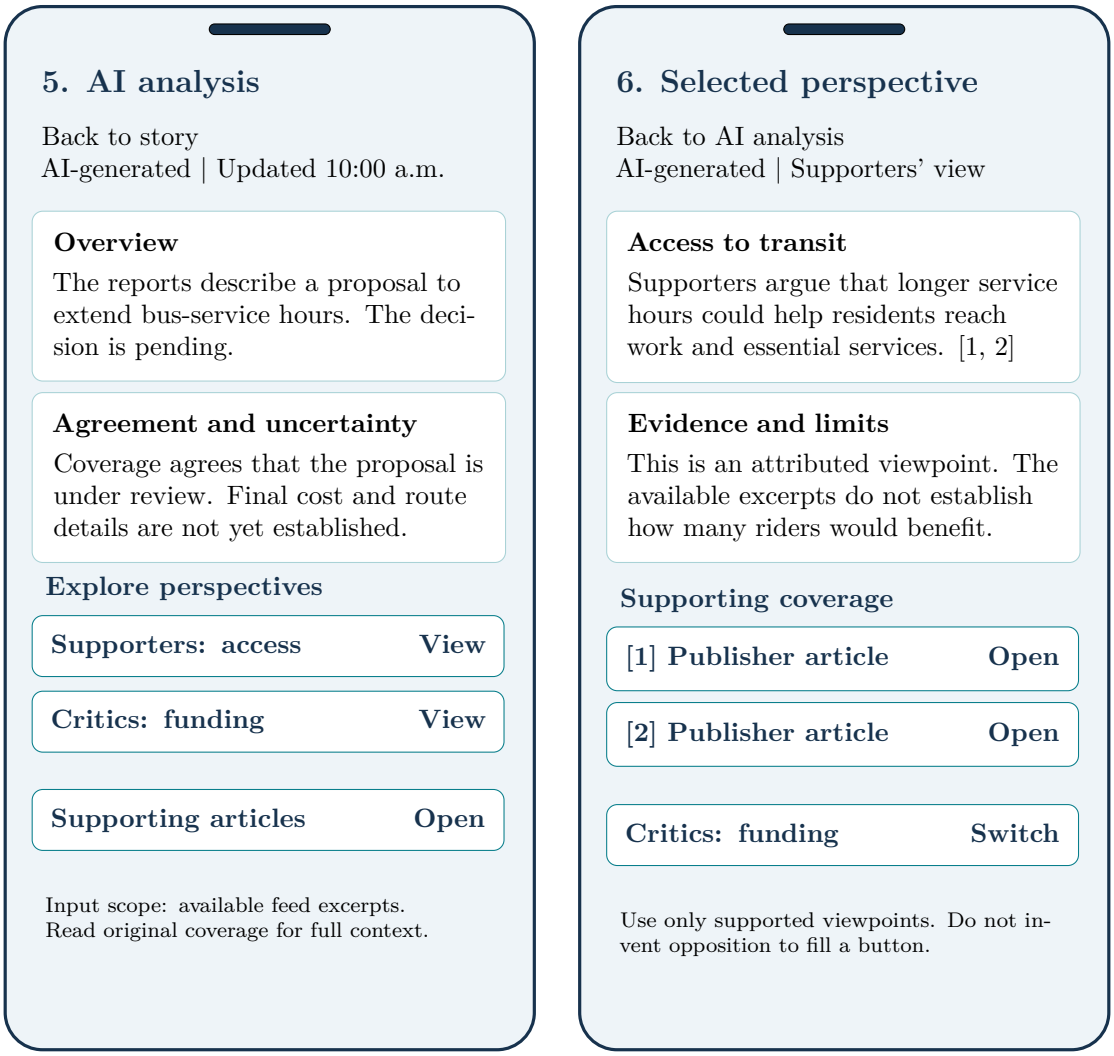
Illustrative wireframes, not a required visual theme. All headlines and article content below are fictional demonstration data; publisher names illustrate button placement.



Interaction: Tap a story to see its detail screen. Publisher buttons open original coverage; background buttons open context. The AI button opens the stored analysis. Story-list refresh contacts your backend only.

Appendix C. Mockups: AI overview and perspectives

AI content must be identified and traceable to the input articles. These examples illustrate the layout, not actual reporting or model output.



Fallback: If the evidence does not support distinct perspectives, show "Insufficient evidence for distinct perspectives." If generation fails, show a clear status while preserving access to original sources.